

A colorful illustration for an Easter egg hunt. It features a large, semi-transparent circle in the center. Inside the circle, there are stylized mountains with white peaks and a yellow path leading through green bushes. The background is divided into three horizontal bands: grey at the top, yellow in the middle, and pink at the bottom. Scattered throughout are various Easter eggs in colors like orange, yellow, white, green, and purple, some with patterns. Small white dots are also scattered across the background.

לה"ו

דו"ח פרוייקט במבוא להנדסת תוכנה

מגישות רינת שטרן 206642159

והילה-שחר חייק 318694403

מטרת הפרויקט הינה יצירת תוכנית לדימוי סצנה גרפית וירטואלית תלת-ממדית. הפרויקט נכתב בשפת JAVA תוך שמירה על עבודה לפי עקרונות עיצוב והנדסת תכנה הנלמדו בקורס התאורטי.

סצנה :

יצירת סצנה של נוף המורכבת מהרים עם שלג עליהם כאשר כל הר מורכב מארבעה משולשים וגם כל חלק של השלג.

בתים קטנים הצופים אל הים – כל בית מורכב מ4 פאות המורכבות כל אחת מ2 משולשים ו4 משולשים המשמים לגג.

בנוסף יש שיחים ירוקים על שפת המים המורכבים מספרות ועננים בשמיים, כל ענן מרכב מ3 ספרות כך שהספירה האמצעית גדולה יותר.

ניתן לראות ספרות של כדורי שלג היורדים מהשמיים ושמש המתחילה לשקוע בין ההרים.

יש לנו בסצנה חמישה מקורות אור :

SpotLight שמאיר את ההרים והעצים בסצנה, במיוחד את החלק המרכזי של הסצנה.

SpotLight שמכוון אל מרכז הסצנה ומדגיש את ההשתקפות במים ואת האובייקטים המרכזיים, כמו השמש וההרים.

DirectionalLight שמספק תאורה כללית לכל האובייקטים בסצנה בצורה אחידה.

DirectionalLight שמאיר את הסצנה מהצד הימני העליון, מדגיש צללים ויוצר השתקפויות על פני המים.

AmbientLight: שמספק תאורה רכה ועדינה לכל הסצנה, מוודא שאין אזורים חשוכים לחלוטין ומוסיף איזון כללי לתאורה.

מטרת השיפור ה-1 של המיני פרויקט:

שיפור איכות הצללים בתמונה באמצעות מתן מראה מציאותי יותר עם צללים רכים בקצוות

נושא: צללים רכים (Soft Shadows)

הבעיה:

במהלך פיתוח מערכת הרינדור, זוהתה בעיה במראה הצללים בתמונות שנוצרו. הצללים נראו לא אמיתיים מכיוון שהיו אחידים בכל האזורים המוצלים, ללא הדרגה טבעית המאפיינת צל במציאות. הצל הקשיח (Hard Shadow) שנוצר נתן תחושה לא מציאותית וגרם לתמונה להיראות מלאכותית.

פתרון:

כדי לפתור את הבעיה ולשפר את מראה הצללים, הוחלט ליישם טכניקה של צללים רכים (Soft Shadows). המרכזי בטכניקה זו הוא להתייחס למקור האור לא כנקודה בודדת במרחב, אלא כשטח בעל מספר נקודות (Area Light). כלומר, במקום לחשב את הצל שנוצר על ידי קרן אור בודדת, נחשב את הצללים שנוצרים כאשר מספר קרניים נשלחות ממקור האור עם פיזור על פני שטח מסוים.

מימוש:

1. הרחבת מודל מקור האור:

- הוספנו למקור האור מאפיין של רדיוס, המייצג את מקור האור כעיגול במרחב (במימוש שלנו השתמשנו בריבוע עבור הפשטות). הרדיוס הזה מאוחסן ב Pointlight בלבד מכיוון שרק שם הוא משפיע.

דוגמא בקוד-

```
private Double3 calcSoftShadows(GeoPoint gp, Vector n, LightSource light) {  
    // יצירת קרן של וקטורים שנשלחת ממקור האור לנקודה על האובייקט  
    List<Vector> vecs2light = light.getRayBeam(gp.point);  
  
    // משתנה לאגירת סכום השקיפות של כל קרן ביחס למקור האור  
    Double3 sumKtr = Double3.ZERO;  
  
    // לולאה שעוברת על כל וקטור בקרן  
    for (Vector l : vecs2light) {  
        // יצירת קרן אור מנקודת הגיאומטריה לכיוון מקור האור, מנוגד לכיוון הווקטור ל  
        Ray lightRay = new Ray(gp.point, l.scale(-1), n); // מהנקודה לכיוון מקור האור  
  
        // חישוב החיתוכים בין קרן האור לאובייקטים בסצנה  
        var intersections = scene._geometries.findGeoIntersections(lightRay, light.getDistance(gp.point));  
  
        // התחלת ההנחה שכל קרן עוברת בצורה מלאה (שקיפות מלאה)  
        Double3 ktr = Double3.ONE;  
  
        // אם אין חיתוכים, הקרן לא חסומה ולכן נשמרת השקיפות המלאה  
        if (intersections == null) {  
            sumKtr = sumKtr.add(ktr); // הוספת השקיפות לסכום  
            continue; // המשך ללולאה הבאה  
        }  
  
        // לולאה שעוברת על כל נקודת חיתוך ובודקת את השקיפות שלה  
        for (GeoPoint p : intersections) {  
            ktr = ktr.product(p.geometry.getMaterial().kT); // חישוב שקיפות מצטברת  
            if (ktr.lowerThan(MIN_CALC_COLOR_K)) { // אם השקיפות נמוכה מערך המינימום  
                ktr = Double3.ZERO; // הקרן נחשבת חסומה לגמרי  
                break; // יציאה מהלולאה כי כבר אין צורך לבדוק עוד נקודות חיתוך  
            }  
        }  
        sumKtr = sumKtr.add(ktr); // הוספת השקיפות לסכום הכולל  
    }  
}
```

```
// אם אין חיתוכים, הקרן לא חסומה ולכן נשמרת השקיפות המלאה  
if (intersections == null) {  
    sumKtr = sumKtr.add(ktr); // הוספת השקיפות לסכום  
    continue; // המשך ללולאה הבאה  
}  
  
// לולאה שעוברת על כל נקודת חיתוך ובודקת את השקיפות שלה  
for (GeoPoint p : intersections) {  
    ktr = ktr.product(p.geometry.getMaterial().kT); // חישוב שקיפות מצטברת  
    if (ktr.lowerThan(MIN_CALC_COLOR_K)) { // אם השקיפות נמוכה מערך המינימום  
        ktr = Double3.ZERO; // הקרן נחשבת חסומה לגמרי  
        break; // יציאה מהלולאה כי כבר אין צורך לבדוק עוד נקודות חיתוך  
    }  
}  
sumKtr = sumKtr.add(ktr); // הוספת השקיפות לסכום הכולל  
}
```

2. חישוב מישור ההכלה:

- כדי למצוא נקודות בתוך העיגול שמייצג את מקור האור, יש להגדיר מישור שבו העיגול מוכל. לשם כך, מצאנו שני וקטורים המוכלים במישור, בדומה להגדרה של מישור ב-Plane View.

באמצעות שני וקטורים אלה ניתן לזוז בתוך המישור ולמצוא כל פעם נקודה אחרת בתוך העיגול המייצג את אזור מקור האור.

דוגמא מהקוד:

```
@Override 1 usage  Rinat Shteren +1*
public List<Vector> getRayBeam(Point p) {
    Vector vec2light = getL(p);
    if (this.radius == 0)
        return List.of(vec2light);

    Vector vec1 = vec2light.makePerpendicularVector();
    Vector vec2 = vec2light.crossProduct(vec1);

    List<Vector> vecs2light = new LinkedList<>();

    for (double i = -radius; i < radius; i += radius / 10) {
        for (double j = -radius; j < radius; j += radius / 10) {
            try {
                vecs2light.add(p.subtract(position.add(vec1.scale(i)).add(vec2.scale(j))));
            } catch (IllegalArgumentException e) {
                vecs2light.add(vec2light);
            }
        }
    }
    return vecs2light;
}
```

3. שליחת קרני צל:

- לאחר שקבענו את הנקודות במישור שמייצג את מקור האור, נשלחים מספר קרני צל מהגוף המוצל לעבר נקודות אלו.
- עבור כל קרן צל, נבדק האם היא פוגעת במקור האור (כלומר, לא נחסמה על ידי גוף אחר). לאחר מכן, מחושב אחוז התאורה בכל נקודה מוצלת על פי היחס בין מספר הקרניים שפגעו במקור האור לבין כלל קרני הצל שנשלחו.
- השיטה הזו מאפשרת ליצור צל רך יחסית, (Soft Shadow) בניגוד לצל הקשיח (Hard Shadow) שבו כל הנקודות המוצלות מקבלות את אותה עוצמת צל.

דוגמא בקוד-

```
private Color calcLocalEffects(GeoPoint gp, Ray ray, Double3 k) { 1usage 1 Rinat Shteren +1 *
    Color color = gp.geometry.getEmission(); // קבלת הצבע הפליטה של הגיאומטריה

    Vector n = gp.geometry.getNormal(gp.point); // חישוב הנורמל לנקודה על האובייקט
    Vector v = ray.getDirection(); // כיוון הקרן
    double nv = alignZero(n.dotProduct(v)); // חישוב מכפלת נקודתית בין הנורמל לכיוון הקרן
    if (nv == 0) return color; // אם הנורמל והכיוון ניצבים זה לזה, אין צורך לחשב תאורה

    Material material = gp.geometry.getMaterial(); // קבלת החומר של האובייקט
    for (LightSource lightSource : scene.lights) { // לולאה על כל מקורות התאורה בסצנה
        Vector l = lightSource.getL(gp.point); // כיוון האור מהנקודה למקור האור
        double nl = alignZero(n.dotProduct(l)); // מכפלת נקודתית בין הנורמל לכיוון האור

        if (nl * nv > 0) { // בדיקה אם מקור האור מאיר על הנקודה ולא מצידה השני
            Double3 ktr = softShadows ? calcSoftShadows(gp, n, lightSource) : transparency(gp, l, n, lightSource);
            // אם משתמשים בצל רך, מחשבים את השקיפות בעזרת calcSoftShadows, אחרת מחשבים שקיפות רגילה

            if (!ktr.product(k).lowerThan(MIN_CALC_COLOR_K)) { // אם השקיפות מספיקה כדי לתרום לצבע הסופי
                Color il = lightSource.getIntensity(gp.point).scale(ktr); // עוצמת האור הנקלטת בנקודה
                color = color.add(il.scale(calcDiffusive(material, nl).add(calcSpecular(material, n, l, nl, v))));
                // חישוב הצבע המקומי על ידי הוספת ההחזרה הדיפוזית וההחזרה המבריקה
            }
        }
    }

    return color; // החזרת הצבע המחושב
}
```

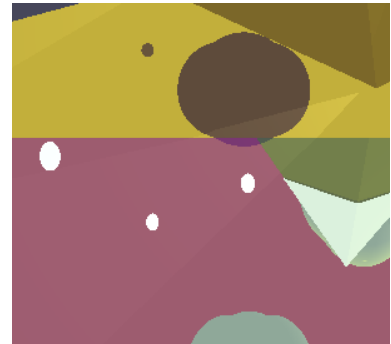
יתרונות השיטה:

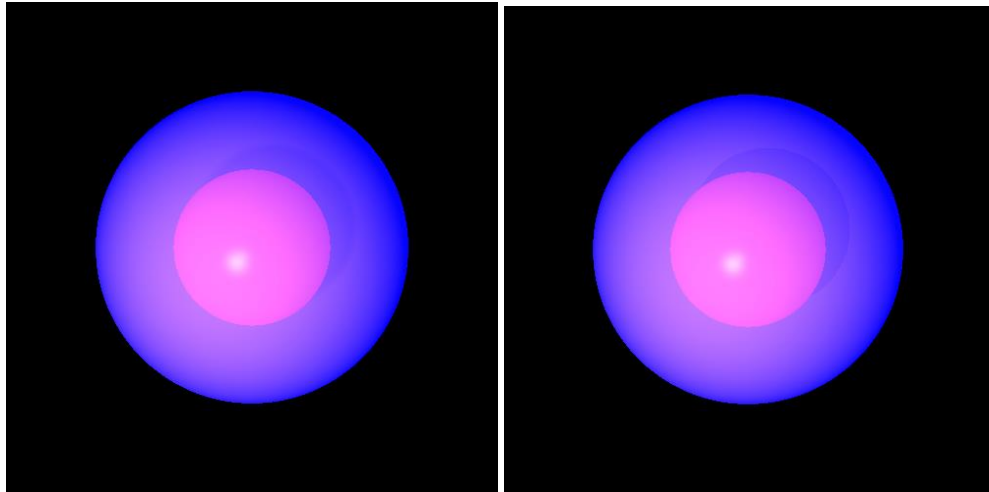
הצל המתקבל הוא בעל הדרגתיות ועומק, מה שתורם למראה מציאותי יותר של התמונה.

ניתן לראות את ההבדלים לאחר השיפור SOFT SHADOW:

תמונה לאחר השיפור:

תמונה לפני השיפור:





שיפור Anti-Aliasing - בונס

מטרה: השגת תמונה חלקה יותר עם מראה מקצועי באמצעות Anti-Aliasing

הבעיה:

כאשר יצרנו את התמונות, התגלתה בעיה של חספוס בשולי הצורות בתמונה. חספוס זה נובע מחישוב הצבע של הפיקסל רק לפי מרכזו, מה שגורם לצורות שאינן ישרות להיראות מחוספסות. תופעה זו משפיעה על איכות התמונה ומבליטה את החלוקה לפיקסלים, מה שיוצר תמונה פחות מציאותית.

הפתרון:

השימוש בטכניקת Anti-Aliasing מתבצע באמצעות שליחת מספר קרניים דרך הפיקסל (בפונקציה `constructRays`) ושקלול הצבעים המוחזרים מכל נקודות הפגיעה (בפונקציה `TraceRays`)

כך מתקבל מראה חלק יותר של הגבולות ככל שמספר הקרניים גדל.

תיאור המימוש:

דוגמה בקוד-

הפונקציה `constructRays()` אחראית על יצירת מספר קרניים עבור כל פיקסל בתמונה. הקרניים נשלחות ממרכז הפיקסל וכן מנקודות רנדומליות בתוכו, על מנת לאפשר חישוב ממוצע מדויק יותר של הצבעים וליצור גבולות חלקים יותר בין הצורות בתמונה.

```

public List<Ray> constructRays(int nX, int nY, int j, int i) { 1 usage  Rinat Shteren *
    List<Ray> rays = new LinkedList<>();
    Point centralPixel = getCenterOfPixel(nX, nY, j, i);
    double rY = _height / nY / antiAliasing;
    double rX = _width / nX / antiAliasing;

    // Variables to store the X and Y offsets of each sub-pixel within the anti-aliasing grid
    double x, y;

    for (int rowNumber = 0; rowNumber < antiAliasing; rowNumber++) {
        for (int colNumber = 0; colNumber < antiAliasing; colNumber++) {
            // Calculate the X and Y offsets for the current sub-pixel
            y = -(rowNumber - (antiAliasing - 1d) / 2) * rY;
            x = (colNumber - (antiAliasing - 1d) / 2) * rX;

            // Calculate the position of the current sub-pixel within the pixel
            Point pIJ = centralPixel;
            if (y != 0) pIJ = pIJ.add(up.scale(y));
            if (x != 0) pIJ = pIJ.add(right.scale(x));

            // Construct a ray from the camera position to the current sub-pixel
            rays.add(new Ray(Plocation, pIJ.subtract(Plocation)));
        }
    }
    return rays;
}

```

דוגמה נוספת בקוד:

הפונקציה `castRay()` בודקת אם נעשה שימוש ב-Anti-Aliasing ואם כן, שולחת קרניים נוספות לכל פיקסל ומחשבת את הצבע הממוצע שלהן. במידה ולא נעשה שימוש בטכניקה זו, היא מחשבת את הצבע של הקרן הרגילה שנשלחת דרך מרכז הפיקסל.

```

private void castRay(int nX, int nY, int col, int row) { 2 usages  Rinat Shteren *
    Color color = Color.BLACK;
    if (antiAliasing > 1) {
        color = SuperSampling(nX, nY, col, row, antiAliasing, adaptiveAliasing: false);
    } else if (adaptive) {
        List<Ray> rays = constructRays(nX, nY, col, row);
        color = rayTracer.TraceRays(rays);
    } else {
        color = rayTracer.traceRay(constructRay(nX, nY, col, row));
    }
    imageWriter.writePixel(col, row, color);
    pixelManager.pixelDone();
}

```

יתרונות השימוש ב-Anti-Aliasing-

- שימוש ב-Anti-Aliasing מאפשר ליצור תמונה חלקה יותר, ללא גבולות חדים בין הפיקסלים, מה שמוביל לתוצאה ויזואלית מציאותית ואיכותית יותר.

- הגבולות "הרכים" יותר של הצורות גורמים לתמונה להראות פחות ממוחשבת ויותר מציאותית, דבר חשוב במיוחד בסצנות מורכבות.

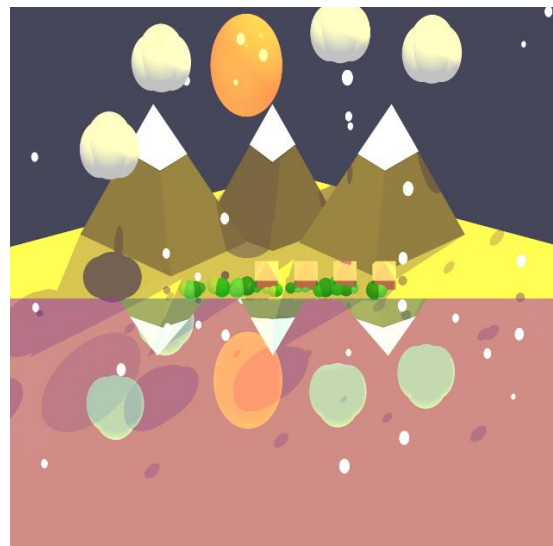
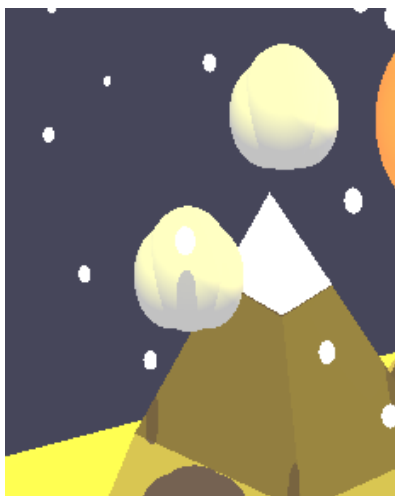
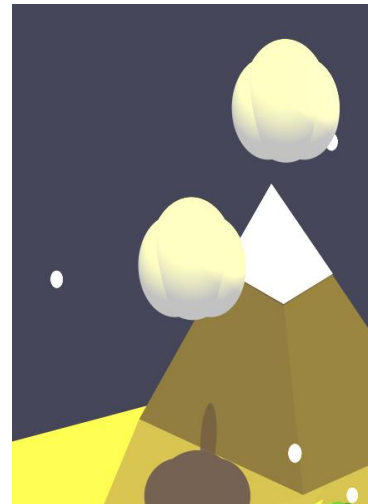
סיכום:

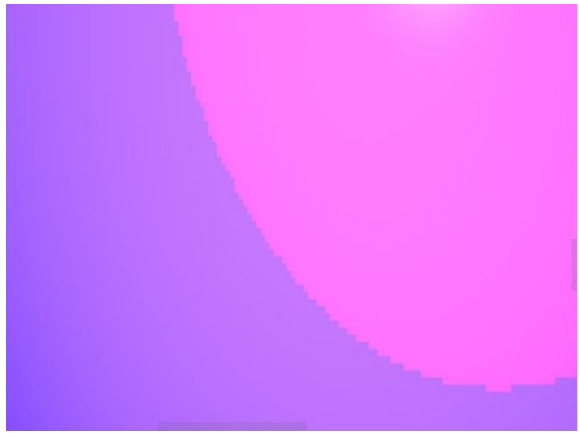
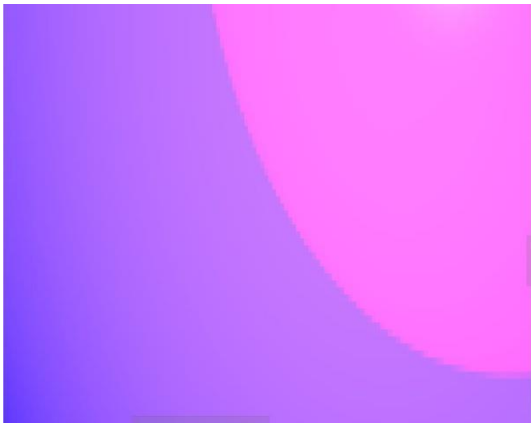
השימוש בטכניקת Anti-Aliasing ושיפור שולי הצורות בתמונה מאפשר להפיק תמונות באיכות גבוהה יותר, עם גבולות רכים יותר, שמובילות לחוויה ויזואלית מציאותית ומשכנעת יותר. הטמעת הפתרון בפרויקט מצליחה לצמצם את החספוס של הקצוות ומשפרת את החלקות הכללית של התמונות.

ניתן לראות את ההבדלים לאחר השיפור **Anti-Aliasing** :

תמונה לפני השיפור:

תמונה לאחר השיפור:





מטרת השיפור ה-2 של המיני פרויקט: שיפור ביצועים

מטרה: שיפור ביצועי התמונה עם תמיכה ב-Adaptive Super Sampling

הבעיה:

כאשר הרצנו את הסצנה עם מספר רב של קרניים לצורך רינדור מדויק יותר, זמן ההרצה עלה בצורה דרסטית. ככל ששלחנו יותר קרניים, הזמן הנדרש לרינדור עלה בצורה ניכרת. מצב זה דורש פתרון יעיל כדי להקטין את זמן הרינדור תוך שמירה על איכות תמונה גבוהה.

הפתרון:

בפרויקט, השתמשנו בשיטת AdaptiveSuperSampling, השיטה מתמקדת במזעור מספר הקרניים שנשלחות לכל פיקסל, תוך שמירה על רמת דיוק גבוהה, על ידי חלוקה רקורסיבית של הפיקסל לאזורים קטנים יותר ורינדור לפי הצורך.

שלבי המימוש:

1. שליחת הפרמטרים לפונקציית -AdaptiveSuperSampling

הפונקציה SuperSampling קובעת את אופן הדגימה של הפיקסל על ידי חישוב הפרמטרים הנדרשים, כמו מספר הקרניים, מרכז הפיקסל וגדלי תת-הפיקסלים. לאחר מכן היא מפעילה את אחת מהשיטות: דגימה רגילה או דגימה אדפטיבית, בהתאם להגדרות. דגימה רגילה מבוצעת אם מספר הקרניים הוא 1, בעוד שדגימה אדפטיבית נבחרת כאשר הפרמטר adaptiveAliasing הוא true.

קטע קוד לדוגמה:

```
private Color SuperSampling(int nX, int nY, int j, int i, int numOfRays, boolean adaptiveAliasing) {
    // קבלת הוקטורים ימינה ולמעלה של המצלמה
    Vector Vright = right;
    Vector Vup = up;

    // קבלת המיקום של המצלמה
    Point cameraLoc = this.getLocation();

    // מספר העמודות בפיקסלים בתמונה (רזולוציה בכיוון האופקי)
    // מספר השורות בפיקסלים בתמונה (רזולוציה בכיוון האנכי)
    // אינדקס העמודה של הפיקסל הנוכחי שבו דוגמים
    // אינדקס השורה של הפיקסל הנוכחי שבו דוגמים
    // מספר הקרניים שנשלחות לכל פיקסל לצורך דגימה
    // adaptiveAliasing: משתנה בוליאני שמצביע האם לבצע סופר-סמפלינג אדפטיבי או רגיל

    // חישוב מספר הקרניים בכל שורה ועמודה בתוך הפיקסל
    int numOfRaysInRowCol = (int) Math.floor(Math.sqrt(numOfRays));

    if (numOfRaysInRowCol == 1)
        return rayTracer.traceRay(constructRayThroughPixel(nX, nY, j, i));

    // חישוב נקודת המרכז של הפיקסל הנוכחי
    Point pIJ = getCenterOfPixel(nX, nY, j, i);

    // חישוב יחס הגובה של הפיקסל (rY)
    double rY = alignZero(number: _height / nY);
    // חישוב יחס הרוחב של הפיקסל (rX)
    double rX = alignZero(number: _width / nX);

    // חישוב יחס הגובה והרוחב של תת-הפיקסלים בתוך הפיקסל
    double PRy = rY / numOfRaysInRowCol;
    double PRx = rX / numOfRaysInRowCol;
```

```
// אם בוצע סופר-סמפלינג אדפטיבי, מפעילים את הפונקציה המתאימה לכך
if (adaptiveAliasing)
    return rayTracer.AdaptiveSuperSamplingRec(pIJ, rX, rY, PRx, PRy, cameraLoc, Vright, Vup, prePoints: null);
else
    // אחרת, מבצעים סופר-סמפלינג רגיל
    return rayTracer.RegularSuperSampling(pIJ, rX, rY, PRx, PRy, cameraLoc, Vright, Vup, prePoints: null);
}
```

2. בדיקה ראשונית של גודל הפיקסל:

- השיטה בודקת אם גודל הפיקסל קטן פי שניים מהגודל המינימלי שנקבע. אם כן, עוקבים אחרי קרן אחת דרך הפיקסל ומחזירים את הצבע שנוצר מבלי להמשיך לחלק את הפיקסל.

קטע קוד לדוגמה:

```
@Override 2 usages Rinat Shteren *
public Color AdaptiveSuperSamplingRec(Point centerP, double Width, double Height, double minWidth, double minHeight,
    Point cameraLoc, Vector Vright, Vector Vup, List<Point> prePoints) {
    if (Width < minWidth * 2 || Height < minHeight * 2) {
        // If the pixel is smaller than the minimum size, trace a ray through the pixel and return the color.
        return this.traceRay(new Ray(cameraLoc, centerP.subtract(cameraLoc)));
    }
}
```

3. חלוקה לתת-פיקסלים:

- אם הפיקסל גדול מהגודל המינימלי, השיטה מחלקת את הפיקסל לארבעה תת-פיקסלים, על ידי חישוב נקודות הפינות והמרכז. נשלחות קרניים דרך כל תת-פיקסל כדי לקבל את הצבעים שלהם.

4. בדיקת דגימה קודמת:

- השיטה בודקת אם תת-פיקסלים כבר נדגמו בעבר כדי למנוע יתירות. אם תת-פיקסל לא נדגם, הוא נוסף לרשימת נקודות המרכז הבאות ונתון לדגימה רקורסיבית נוספת.

קטע קוד לדוגמה:

```
List<Point> nextCenterPList = new LinkedList<>();
List<Point> cornersList = new LinkedList<>();
List<primitives.Color> colorList = new LinkedList<>();
Point tempCorner;
Ray tempRay;
// Iterate over the corners of the pixel and perform sub-sampling
for (int i = -1; i <= 1; i += 2) {
    for (int j = -1; j <= 1; j += 2) {
        tempCorner = centerP.add(Vright.scale( scalar: i * Width / 2)).add(Vup.scale( scalar: j * Height / 2));
        cornersList.add(tempCorner);
        // Check if the sub-pixel's corner is already sampled
        if (prePoints == null || !isInList(prePoints, tempCorner)) {
            tempRay = new Ray(cameraLoc, tempCorner.subtract(cameraLoc));
            nextCenterPList.add(centerP.add(Vright.scale( scalar: i * Width / 4)).add(Vup.scale( scalar: j * Height / 4)));
            colorList.add(traceRay(tempRay));
        }
    }
}
```

5. חישוב צבע הפיקסל:

- השיטה בודקת אם כל תת-הפיקסלים מניבים את אותו צבע. אם כן, היא מחזירה את הצבע כתוצאה סופית לפיקסל. אם לא, היא ממשיכה בחלוקה רקורסיבית של תת-הפיקסלים.

קטע קוד לדוגמה:

```

boolean isAllEquals = true;
primitives.Color tempColor = colorList.get(0);
// Check if all colors in the colorList are almost equal
for (primitives.Color color : colorList) {
    if (!tempColor.isAlmostEquals(color))
        isAllEquals = false;
}
if (isAllEquals && colorList.size() > 1)
    // If all colors are equal and there is more than one color, return the first color.
    return tempColor;

```

6. סיכום הצבעים:

- לבסוף, השיטה מחזירה את הצבע הממוצע של כל תת-הפיקסלים, אשר מחושב על ידי חלוקתו במספר תת-הפיקסלים.

קטע קוד לדוגמה:

```

tempColor = primitives.Color.BLACK;
// Recursively perform adaptive super-sampling on sub-pixels
for (Point center : nextCenterPList) {
    tempColor = tempColor.add(AdaptiveSuperSamplingRec(center, Width: Width / 2, Height: Height / 2, minWidth,
        minHeight, cameraLoc, Vright, Vup, cornersList));
}
// Reduce the color by dividing by the number of sub-pixels
return tempColor.reduce(nextCenterPList.size());
}

```

ביצועים משופרים:

טכניקת דגימת העל האדפטיבית מאפשרת עיבוד יעיל יותר על ידי מיקוד משאבי חישוב באזורים הדורשים דגימה נוספת, ובכך מקצרת את זמן הרינדור מבלי להתפשר על איכות התמונה.

SoftShadow יחד עם MultiThreading AdaptiveSuperSampling –

✓ Tests passed: 1 of 1 test – 58 sec 778 ms

SoftShadow בלבד-

✓ Tests passed: 1 of 1 test – 1 min 10 sec

AntiAliasing MultiThreading, AdaptiveSuperSampling –

✓ ReflectionRefractionTest 3 min 18 sec
 ✓ twoSpheres() 3 min 18 sec

בתמונה של הכדור

בתמונה הסופית שלנו

✓	test_FINAL_anti_analizing (renderer)	5 hr 55 min
✓	ourPicture()	5 hr 55 min

AntiAliasing בלבד-

✓	ReflectionRefractionTest	2 min 55 sec
✓	twoSpheres()	2 min 55 sec